

A Semantic-Layer-Mediated Agent for Natural Language to SQL over Heterogeneous Enterprise Databases

^{1st} Ha Jeong Kim
DAQUV Corp.
Seoul, Republic of Korea
hajkim@daquv.com

^{2nd} Saksonita Khoeurn
Dept. of Management Information Systems
Chungbuk National University
BigDataLabs, Co., Ltd.
Cheongju, Republic of Korea
saksonita@chungbuk.ac.kr

^{3rd} Ye Ji Yoon
DAQUV Corp.
Seoul, Republic of Korea
philoyyj@daquv.com

Abstract—Natural language to SQL (NL2SQL) over real enterprise databases remains substantially harder than over academic benchmarks: schemas contain hundreds of physical tables with opaque column names, dialects differ across engines, and a single analytical question may require nested aggregation, time-windowed logic, and multi-table joins. Directly prompting a large language model (LLM) with raw schema text exposes the model to this complexity all at once and yields brittle queries. This paper presents the architecture of a semantic-layer-mediated NL2SQL agent that decouples *intent* from *physical execution*. Rather than asking the LLM to write SQL against raw tables, the agent reasons over a curated semantic layer through a compact intermediate representation we call the Semantic Model Query (SMQ); a deterministic engine compiles SMQs into dialect-correct SQL, which the agent then inspects, composes, and executes. The system follows a single-tool think-act loop, routes execution across SQLite, BigQuery, and Snowflake backends, and is packaged as an end-to-end evaluation harness. Driven by Gemini 3 Pro, the system attains 94.15% execution accuracy on the 547-task Spider2-snow benchmark—a suite of real-world enterprise NL2SQL tasks executed against Snowflake—the third-highest entry on the official leaderboard and far above schema-only baselines. We describe the component design, the SMQ representation, the agent’s exploration strategy, and the per-backend results, and we discuss the quality and overfitting tensions inherent to maintaining a semantic layer as a context source for an LLM agent.

Index Terms—text-to-SQL, NL2SQL, large language model agents, semantic layer, intermediate representation, tool use, enterprise databases

I. INTRODUCTION

Translating natural language questions into executable SQL (NL2SQL) is a long-standing goal for democratizing data access. Recent large language models (LLMs) achieve strong results on academic benchmarks such as Spider [2] and BIRD [3], where schemas are small and questions map fairly directly to single queries. Enterprise settings are different in kind, not merely in degree. The Spider2 benchmark [1] was introduced precisely to capture this gap: its tasks operate over

production data warehouses with hundreds of columns, cryptic naming conventions, dialect-specific functions, and questions whose gold answers routinely span dozens of lines of SQL with common table expressions (CTEs), window functions, and multi-step aggregation. Reported accuracies on Spider2 are far below the near-saturation numbers seen on Spider, confirming that “put the schema in the prompt and ask for SQL” does not transfer to real workloads.

We argue that the difficulty has two distinct sources. The first is *grounding*: the model must discover which physical tables and columns are relevant and how they join, out of a large and poorly self-describing schema. The second is *composition*: the model must assemble correct, dialect-valid SQL once the right building blocks are known. Conflating these two problems in a single free-form generation step is what makes naive prompting brittle—a single wrong column name or join predicate fails the whole query, and the model has no structured surface on which to recover.

This paper presents the architecture of *spider2-daquv-quvi*, an NL2SQL agent that separates grounding from composition by interposing a *semantic layer* between the LLM and the database. The semantic layer is a curated, business-oriented description of each database: tables are wrapped as *semantic models* exposing named *dimensions*, *measures*, and *metrics*, with human-readable descriptions and the physical column expressions they map to. The agent never sees raw schema first; instead it issues a compact, structured query—the Semantic Model Query (SMQ)—against the semantic layer. A deterministic engine compiles each SMQ into dialect-correct SQL and returns it. The agent uses these compiled fragments as *verified building blocks*: it inspects the physical column expressions and join patterns they reveal, composes them into a final query (adding constructs the SMQ compiler does not support, such as window functions or recursive CTEs), and executes that query against the appropriate backend.

The contributions of this paper are:

- A system architecture for NL2SQL that mediates LLM reasoning through a semantic layer and a structured

intermediate representation (SMQ), decoupling schema grounding from SQL composition (Sections III–IV).

- A single-tool think-act agent loop in which SMQ compilation is used for *exploration* and direct SQL execution is the single terminal action, together with the prompting strategy that enforces this discipline (Section V).
- A multi-backend execution and evaluation harness that routes queries to SQLite, BigQuery, and Snowflake and scores them under the Spider2 protocol (Section VI).
- An empirical study on the 547-task Spider2-snow benchmark, reporting 94.15% execution accuracy with a per-backend breakdown and a comparison against published leaderboard methods, and a discussion of the practical tension between semantic-layer quality and overfitting to the evaluation set (Sections VII–VIII).

II. RELATED WORK

Text-to-SQL benchmarks. Spider [2] established cross-domain, multi-table semantic parsing as a standard task; BIRD [3] added larger, dirtier databases and an emphasis on efficiency and external knowledge. Spider2 [1] raised the bar to enterprise-scale warehouses on BigQuery, Snowflake, and local engines, with long, realistic gold queries; it is the benchmark we target. Our work evaluates on the Spider2-snow split.

LLM methods for text-to-SQL. Prompting and decomposition methods such as DIN-SQL [4], DAIL-SQL [5], and C3 [6] improve single-shot generation through schema linking, few-shot selection, and self-correction. Multi-agent and pipeline systems such as MAC-SQL [7] and CHESS [8] introduce specialized decomposer/selector/refiner roles and schema-pruning stages. These approaches operate directly on physical schemas; our system instead routes reasoning through a curated semantic layer, treating compiled SMQ→SQL fragments as verified grounding evidence rather than relying on the model to recall physical names from a flat schema dump.

Semantic layers. Business-intelligence semantic layers and metrics frameworks (e.g., dbt’s MetricFlow [10]) let analysts define dimensions, measures, and metrics once and reuse them across queries. We repurpose this idea as an *LLM grounding substrate*: the semantic layer is both the model’s view of the data and the source of compilable, dialect-correct SQL.

LLM agents and tool use. The think-act paradigm of ReAct [9] interleaves reasoning traces with tool calls. Our agent adopts a strict single-tool-per-step variant in which one tool (SMQ compilation) is for exploration and another (SQL execution) is the single terminating action, constraining the agent’s action space to reduce error propagation.

III. SYSTEM ARCHITECTURE

Fig. 1 shows the end-to-end architecture. The system is organized into four tiers: (1) an *orchestration* tier that loads benchmark instances and drives batched, parallel execution; (2) the *QUVI* NL2SQL service that hosts the LLM agent and the semantic layer; (3) a deterministic *SMQ-to-SQL engine*; and (4) a *multi-backend executor* that runs the final SQL

against the correct database. Each instance is processed as an independent workflow keyed by an instance identifier and timestamp.

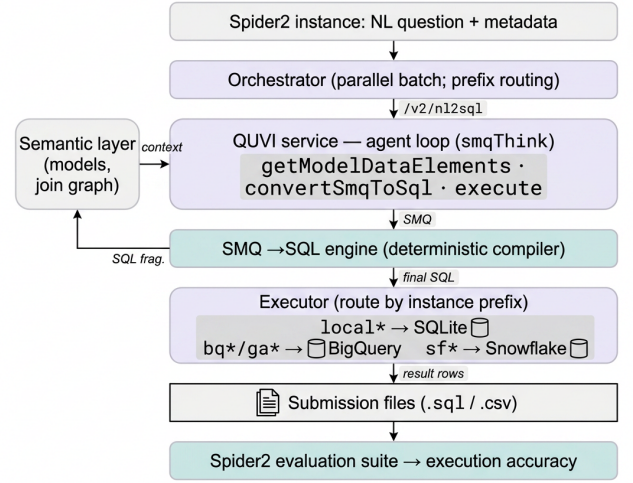


Fig. 1. End-to-end architecture. The orchestrator dispatches NL questions to the QUVI service, which runs the agent loop over the semantic layer. SMQs are compiled to SQL by the engine; the final SQL is routed by instance prefix to the SQLite, BigQuery, or Snowflake executor. Results are written to submission files and scored by the Spider2 evaluation suite.

A. Orchestration Tier

The entry point parses a run specification—database split (*lite/snow*), and an instance selector (explicit IDs, ranges, prefixes, or all). An instance loader reads the Spider2 instance files, which contain the natural-language question and metadata. An execution service then dispatches instances through a thread pool for parallel batches, with backoff on rate limiting (e.g., HTTP 429), per-instance timeouts, and bounded retries. For each instance it mints a workflow identifier and calls the NL2SQL client, which authenticates per database user before posting the question to the QUVI service.

B. QUVI NL2SQL Service

QUVI hosts the agent and the semantic layer. On receiving a question it (i) looks up the relevant semantic models for the target database, (ii) runs the think-act agent loop (Section V) that emits SMQs and SQL, and (iii) delegates SMQ compilation to the engine. The semantic layer is a per-database directory containing source table definitions, a join graph, date and time-spine configuration, and one YAML file per semantic model.

C. SMQ-to-SQL Engine

A separate engine endpoint compiles an SMQ into SQL deterministically (*SmqToSql*). It resolves each referenced dimension/measure/metric to its physical column expression, injects join predicates from the join graph, and emits SQL in the target dialect. This component is the source of *ground truth*

physical names: every table name and column expression the agent uses in its final query is lifted from a compiler output, not hallucinated.

D. Multi-Backend Executor

A custom execution server exposes a single endpoint and routes by instance-ID prefix to the correct backend (Table I). Each executor holds the appropriate credentials/driver and returns results as a list of row dictionaries, which the orchestrator serializes to a SQL file and a CSV for scoring.

TABLE I
BACKEND ROUTING BY INSTANCE IDENTIFIER PREFIX

Prefix	Backend	Executor
local*	SQLite	sqlite_executor
bq*, ga*	BigQuery	bigquery_executor
sf*, sf_bq*	Snowflake	snowflake_executor

IV. THE SEMANTIC LAYER AND SMQ

A. Semantic Models

Each physical table is wrapped by a semantic model that gives it a business name and exposes typed, described data elements. *Dimensions* are aggregation criteria (grouping/filtering keys), *measures* are aggregation targets, and *metrics* are predefined aggregations or derived calculations over measures and dimensions. Crucially, each element carries (a) a human-readable description that the LLM reads during grounding, and (b) an `expr` field holding the exact physical column expression used during compilation. The abstract name and the physical expression are thus kept separate, which lets the agent reason about *intent* while the engine handles *physical mapping*. Listing 1 shows an excerpt.

```
- name: RetailAnalyticsSalesModel
  table: ..._snowflake('RETAIL_ANALYTICS_SALES')
  dimensions:
    - name: asin
      type: varchar
      description: Amazon Standard Identification Number
      expr: ASIN
  measures:
    - name: orderedRevenue
      type: float
      description: ordered revenue
      expr: ORDERED_REVENUE
```

Listing 1. Semantic model excerpt: a dimension and a measure, each pairing a description with a physical expression.

B. The Join Graph

Inter-model joins are declared once in a per-database join graph as typed edges: a `from` model, a `to` model, a join key, and an `on` predicate giving the left/right expressions (including transformations such as `TRIM` on a dirty key). The compiler consults this graph to assemble joins automatically, so the agent does not need to rediscover join predicates for the supported cases.

C. Semantic Model Query (SMQ)

The SMQ is the intermediate representation between the agent and the engine. It is a compact JSON object with three lists—`metrics` (query targets), `filters` (WHERE conditions), and `group_by` (grouping dimensions). Elements are referenced by a uniform naming convention, `ModelName__elementName` for dimensions and measures, and by bare name for metrics. An example SMQ and its role are shown in Listing 2. The SMQ deliberately covers only the common analytical core (selection, filtering, grouping, declared joins); it does not express CROSS JOINS, arbitrary subqueries, advanced window functions, or recursive CTEs. This is by design: the SMQ is an *exploration and grounding* instrument, and the long tail of SQL complexity is handled by the agent composing on top of compiler outputs.

```
{
  "metrics": ["RetailAnalyticsSalesModel__orderedRevenue"],
  "filters": ["RetailAnalyticsSalesModel__period = 'DAILY'"],
  "group_by": ["RetailAnalyticsSalesModel__asin"]
}
```

Listing 2. An SMQ. The engine compiles it to dialect-correct SQL and returns the SQL plus a result preview.

V. AGENT WORKFLOW

A. Think-Act Loop

The agent runs a constrained think-act loop. Each turn it emits a private reasoning block followed by exactly one tool call. Three tools are available: `getModelDataElements` (list the metrics/dimensions of selected models), `convertSmqToSql` (compile an SMQ, returning the SQL and a five-row result preview), and `execute` (run a final SQL query). Restricting each response to a single tool call narrows the action space and makes the trajectory auditable. Algorithm 1 summarizes the loop.

Algorithm 1 Semantic-layer-mediated NL2SQL agent loop

- 1: **Input:** question q , semantic models M for target DB, dialect d
- 2: $E \leftarrow \text{GETMODELDATELEMENTS}(\text{relevant}(M, q))$ ▷ discover elements
- 3: $B \leftarrow \emptyset$ ▷ verified SQL building blocks
- 4: **repeat**
- 5: write an SMQ s for an intermediate result
- 6: $(sql, preview) \leftarrow \text{CONVERTSMQTOSQL}(s)$
- 7: $B \leftarrow B \cup \{(expr/table \text{ names from } sql)\}$
- 8: **until** building blocks suffice for q
- 9: compose final SQL Q from B (add CTEs, window fns, subqueries)
- 10: **return** $\text{EXECUTE}(Q)$ ▷ single terminal action

B. SMQ-for-Exploration Discipline

The defining prompt strategy is that `convertSmqToSql` is used for *exploration*, not for producing the final answer. Its purpose is to reveal how abstract elements map to physical

columns, the dialect’s syntax, and join patterns. The agent extracts physical table names and column expressions from the compiled SQL and assembles its own final query, applying constructs beyond the SMQ’s expressiveness. A strict constraint forbids referencing semantic model names as physical tables in any executed SQL: only names lifted from a compiler output may appear. Direct schema introspection (e.g., querying `INFORMATION_SCHEMA`) is disallowed, forcing all grounding through the semantic layer. The single `execute` call is terminal: a successful execution ends the workflow.

C. Why Mediation Helps

This design addresses both difficulty sources from Section III. Grounding becomes a guided search over described, business-named elements rather than a guess over a flat schema; and every physical identifier the model commits to has already been validated by the compiler, so composition errors are confined to the structure the agent adds on top, where they are easier to detect and repair via re-exploration.

VI. EXECUTION AND EVALUATION HARNESS

Final SQL is dispatched to the backend executor and the returned rows are written as a per-instance SQL file and CSV under a submission directory, with a human-readable Markdown trace of the full tool-call history saved alongside for debugging. Scoring uses the official Spider2 evaluation suite in two modes: execution-result comparison (the submitted result set is compared against the gold result) and SQL comparison. Configuration (service ports, data and semantic-layer paths, timeouts, retry limits, credentials) is centralized in a single application config so the same harness runs across the SQLite, BigQuery, and Snowflake environments without code changes.

VII. EVALUATION

A. Setup

We evaluate on Spider2-snow, a benchmark of 547 enterprise NL2SQL instances executed against Snowflake, spanning databases of SQLite-, BigQuery-, GA4-, and native-Snowflake origin. The underlying LLM is Gemini 3 Pro (preview), invoked through the semantic-layer `smqThink` agent node at temperature 0.1 with extended thinking enabled at the high setting, a 16,384-token output budget, and a cap of 20 SMQ iterations per instance; the SMQ→SQL compiler is deterministic. We find the agent’s accuracy to be sensitive to the model’s thinking configuration: extended thinking is what allows the model to compose the long, multi-step SQL that Spider2-snow demands, and disabling it substantially degrades accuracy. We report execution accuracy: an instance is correct if the executed query’s result set matches the gold result under the official Spider2 execution-result protocol.

B. Overall Result

On Spider2-snow the system answers 515 of 547 instances correctly, an execution accuracy of **94.15%**. Given that Spider2 gold queries are long, multi-step, and dialect-specific, this result indicates that semantic-layer mediation, paired

with a strong reasoning model and well-curated per-database semantic layers, provides highly effective grounding on real enterprise schemas.

C. Per-Backend Breakdown

Table II breaks accuracy down by instance class. Accuracy exceeds 94% on every database-origin class except native Snowflake (`sf`, 55.6%), which is also the smallest class (18 instances) and thus the highest-variance. Backend-specific dialect handling and the maturity of each database’s semantic layer—not the agent loop itself—are the dominant factors in the remaining failures.

TABLE II
EXECUTION ACCURACY ON SPIDER2-SNOW BY DATABASE-ORIGIN CLASS (GEMINI 3 PRO)

Instance class (origin)	Correct/Total	Accuracy
<code>sf_local</code> (SQLite-origin)	131 / 135	97.0%
<code>sf_ga</code> (GA4-origin)	24 / 25	96.0%
<code>sf_bq</code> (BigQuery-origin)	350 / 369	94.9%
<code>sf</code> (native Snowflake)	10 / 18	55.6%
Total	515 / 547	94.15%

D. Comparison with Published Methods

Table III places our Spider2-snow result against entries on the official Spider2 leaderboard [12]. General agent and prompting pipelines on raw schemas remain low on this benchmark—DAIL-SQL [5] with GPT-4o reaches only 2.2%, Spider-Agent [1] 23–26% depending on the backbone, and ReFoRCE [11] 31.3%. Our semantic-layer-mediated agent (listed as *QUVI-3 + Gemini-3-pro-preview*) attains **94.15%**, the third-highest entry overall. The large margin over schema-only baselines is consistent with our central claim that mediating the LLM through a curated semantic layer is decisive for enterprise NL2SQL; we note, however, that the semantic layer encodes per-database domain knowledge that the zero-shot baselines do not have access to, so the comparison reflects the value of the *system* (semantic layer plus agent), not the LLM alone.

TABLE III
EXECUTION ACCURACY ON SPIDER2-SNOW: PUBLISHED METHODS (SPIDER2 LEADERBOARD)

Method	Accuracy
DAIL-SQL + GPT-4o [5]	2.20%
Spider-Agent + o1-preview [1]	23.58%
Spider-Agent + Claude-4-Sonnet [1]	25.78%
ReFoRCE + o1-preview [11]	31.26%
Ours (QUVI-3 + Gemini 3 Pro)	94.15%

E. Failure Modes

From the saved execution traces, recurring failure modes include: (i) SMQ compilation gaps, where a referenced model lacks a needed join relation and the compiler cannot assemble the fragment; (ii) composition errors in the agent-authored

portion of complex queries (nested time windows, exact rounding and ranking semantics); and (iii) under-specified semantic layers, where a column needed by the gold query is not yet exposed as a described element. The first and third are addressed by improving the semantic layer rather than the agent, which motivates the discussion below.

VIII. DISCUSSION

A. Semantic-Layer Quality is the Lever

Because grounding is delegated to the semantic layer, system accuracy is strongly bounded by how well each database’s models describe the data and declare its joins. In practice, improving a database’s semantic models—adding missing elements, sharpening descriptions, declaring join edges—directly lifts accuracy on that database. This makes the semantic layer the primary maintenance surface.

B. The Overfitting Tension

Curating the semantic layer against an evaluation set introduces a Goodhart-style risk: descriptions can drift from concise, generalizable annotations toward verbose text that effectively encodes the expected answer for known questions. Such context may raise benchmark accuracy while reducing robustness to unseen questions and schema change—the analogue of reward-proxy overoptimization. We therefore treat the semantic layer as code subject to review: descriptions should encode reusable schema knowledge, not question-specific hints, and structural information (joins, expressions) should live in typed fields rather than prose. Quantifying and regularizing this trade-off—an intrinsic quality signal for constructed context that is independent of downstream task accuracy—is an open problem and a direction for future work.

C. Limitations

The SMQ deliberately covers only a common analytical core, so the hardest queries depend on agent-authored SQL whose correctness the engine does not guarantee. The evaluation is single-benchmark (Spider2-snow) and uses execution-result matching, which can over- or under-credit edge cases. Per-backend results for the smallest classes are high-variance. Finally, accuracy is entangled with the maturity of each database’s hand-authored semantic layer, which varies across databases.

IX. CONCLUSION

We presented the architecture of a semantic-layer-mediated NL2SQL agent that decouples schema grounding from SQL composition. By interposing a curated semantic layer and a compact SMQ intermediate representation between the LLM and the database, and by using deterministic SMQ→SQL compilation as a source of verified building blocks within a constrained single-tool agent loop, the system reaches 94.15% execution accuracy on the 547-task Spider2-snow benchmark—the third-highest entry on the official leaderboard and far above schema-only baselines. The results indicate that mediating LLM reasoning through a structured, business-oriented layer

is an effective strategy for enterprise NL2SQL, and that the central remaining challenge is maintaining semantic-layer quality without overfitting it to the evaluation set.

REFERENCES

- [1] F. Lei, J. Chen, Y. Ye, R. Cao, *et al.*, “Spider 2.0: Evaluating language models on real-world enterprise text-to-SQL workflows,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2025.
- [2] T. Yu, R. Zhang, K. Yang, *et al.*, “Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task,” in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2018, pp. 3911–3921.
- [3] J. Li, B. Hui, G. Qu, *et al.*, “Can LLM already serve as a database interface? A big bench for large-scale database grounded text-to-SQLs (BIRD),” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [4] M. Pourreza and D. Rafiei, “DIN-SQL: Decomposed in-context learning of text-to-SQL with self-correction,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [5] D. Gao, H. Wang, Y. Li, *et al.*, “Text-to-SQL empowered by large language models: A benchmark evaluation,” *Proc. VLDB Endowment*, vol. 17, no. 5, pp. 1132–1145, 2024.
- [6] X. Dong, C. Zhang, Y. Ge, *et al.*, “C3: Zero-shot text-to-SQL with ChatGPT,” arXiv:2307.07306, 2023.
- [7] B. Wang, C. Ren, J. Yang, *et al.*, “MAC-SQL: A multi-agent collaborative framework for text-to-SQL,” in *Proc. Int. Conf. Computational Linguistics (COLING)*, 2025.
- [8] S. Talaei, M. Pourreza, Y.-C. Chang, *et al.*, “CHESS: Contextual harnessing for efficient SQL synthesis,” arXiv:2405.16755, 2024.
- [9] S. Yao, J. Zhao, D. Yu, *et al.*, “ReAct: Synergizing reasoning and acting in language models,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2023.
- [10] dbt Labs, “dbt Semantic Layer and MetricFlow,” Technical documentation, 2024. [Online]. Available: <https://docs.getdbt.com/docs/build/about-metricflow>
- [11] M. Deng, A. Ramachandran, C. Xu, *et al.*, “ReFoRCE: A text-to-SQL agent with self-refinement, consensus enforcement, and column exploration,” arXiv:2502.00675, 2025.
- [12] XLang Lab, “Spider 2.0 leaderboard,” 2026. [Online]. Available: <https://spider2-sql.github.io/>